

UNIVERSITY OF WATERLOO
FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Brew It Yourself

An Automated Single Vessel Home Brewery System

Group Number: 2016.019
Consultant: Douglas Harder



Date: July 2, 2015

This is a very nice report, earning a score of 65.9/80. Strengths include quality of justifications and quantitative analysis in Section 3.1. Upper-year knowledge is used throughout.

Contents

1	High-Level Description of Project	2
1.1	Motivation	2
1.2	Project Objective	2
1.3	Block Diagram	2
1.3.1	Description of System	3
1.3.2	Designing and Not Designing Components	4
2	Project Specifications	5
2.1	Functional Specifications	5
2.2	Non-Functional Specifications	7
3	Detailed Design	8
3.1	Mechanical Design	8
3.1.1	Material	8
3.1.2	Tank Structure	8
3.2	Electrical System	10
3.2.1	Controller Designs	10
3.2.2	Power Electronics Design	15
3.3	Computer System	17
3.3.1	Embedded Computer	17
3.3.2	Sensors and Microcontrollers	18
3.3.3	Web Server and Database	20
3.4	Mobile Application	21
3.4.1	Real Time Data	22
3.4.2	Input Controls and Interaction	23
4	Discussion and Project Timeline	24
4.1	Evaluation of Final Design	24
4.2	Use of Advanced Knowledge	24
4.3	Creativity, Novelty, Elegance	24
4.4	Student Hours	25
4.5	Potential Safety Hazards	25
4.6	Project Time-Line	25
	Acronyms	28
	Beer Terminology	28
	Technical Terminology	28
A	Appendix	30

1 High-Level Description of Project

1.1 Motivation

The art of home brewing has been steadily gaining popularity over the past 35 years alongside the rise of craft breweries in North America, so much so that in 2010 there were over 2000 craft breweries in the United States, after starting with only 8 in 1980 [1]. The traditional method for homebrewing requires various components, constant monitoring and heavy maintenance. There should be a solution which reduces complexity, making it much more affordable and practical for home use. We hope to create a single vessel system that would make the home brewing process precise, automated and compact, all at a reasonable price.

1.2 Project Objective

The objective of this project is to combine homebrewing experience with engineering design, and construct a single vessel brewing system. By maintaining a strict control of key parameters, the brewing process is regulated using a combination of fluid mechanics, heat transfer, digital controls, power systems, embedded robotics and mobile development. The Robotics Operating System (ROS), allows for a design where sensors can be added to a modular setup and provide feedback. By receiving feedback from temperature readings, density measurements and pH monitoring, the brewing process can be accurately recorded, shared, and automated by the system.

1.3 Block Diagram

Figure 1 shows relevant links between the physical, mechanical design of the brewing system, as well as the computer systems and their underlying software.

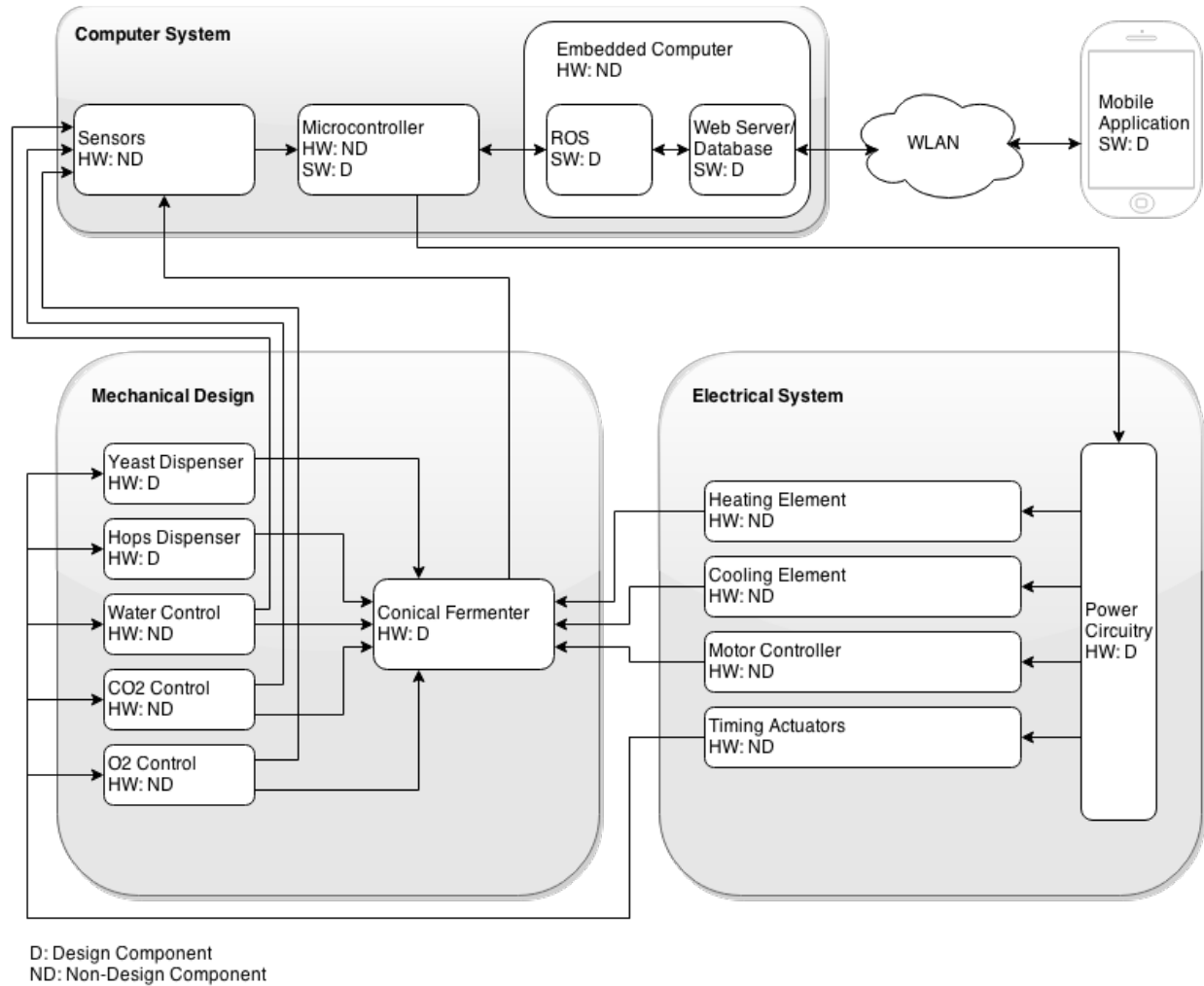


Figure 1: Block Diagram outlining the interactions between the computer, electrical, and mechanical systems

1.3.1 Description of System

The single vessel brewing system, as shown in Figure 1, contains various sensors which forward the environmental parameters to the microcontroller. The data from the various sensors accurately represents the state of the fermenter and feed into digital control loops running on the microcontroller. Data from sensor modules is obtained by a microcontroller via an Inter-Integrated Circuit (I2C) interface. This data is then sent to an embedded computer system over Universal Serial Bus (USB) where it is logged in a database and used to control relevant subsystems. Control commands are sent from the embedded computer to the microcontroller, where the appropriate component can be communicated with in order to regulate the brewing environment. The embedded computer system is then able to send diagnostic information and push notifications to a mobile device through a Wireless Local Area Network (WLAN) connection.

1.3.2 Designing and Not Designing Components

The following is an outline of the components in the block diagram and the requirements associated for each Designing (D) and Non-Designing (ND) component.

- **Sensors**

ND: Sensors being used in the vessel are not being designed. Instead, off-the-shelf sensors are being purchased and integrated into the system. Relevant sensor types for this system include, but aren't limited to, pH sensors, volume sensors, flow meters, temperature probes, etc.

- **Microcontroller(s)** ND: The hardware of the microcontroller is not being designed since that's outside the scope of this project. The system instead uses an off-the-shelf microcontroller. D: The software running on the microcontroller is being designed. This software mainly consists of digital control loops for interfacing with the sensors and power electronic circuitry.

- **Conical Fermentation Vessel**

D: The mechanical features and dimensions of the conical fermentation vessel are being designed to hold various sensors and other electrical and mechanical components.

- **Power Electronic Circuitry**

D: The power electronic circuitry is being designed to power the pumps, motors, actuators, and heating/cooling mechanisms. The circuitry takes input from the digital controller running on the microcontroller and outputs the appropriate power to the end devices.

- **Heating and Cooling Elements**

ND: The heating coils and refrigeration unit used to regulate temperature of the vessel are not being designed. Instead, off-the-shelf or salvaged and adapted components from existing appliances (e.g. hot water tank coils, home air conditioning heat pump) are being used and the power electronic circuitry is being designed to power these devices.

- **Motor Controller**

ND: The motors, pumps, and mixing mechanisms used in our system are not being designed. Instead, off-the-shelf or salvaged and adapted components from existing appliances (e.g. blender mixing prop, aquarium pumps) are being used and the power electronic circuitry is being designed to power these devices.

- **Timing actuators**

ND: The actuators used in our system are not being designed. Instead, off-the-shelf solenoids and servo motors are being used and the mechanical subsystems that they will actuate are being designed. The power electronic circuitry is also being designed to be able to power the actuators.

- **Embedded Computer System**

ND: The hardware and operating system of the embedded computer system is not a design objective as it is outside the scope of this project. Instead an off-the-shelf embedded computer is purchased and a Debian based Linux operating system is installed to satisfy the requirements for the development environment.

D: The embedded computer system is configured with a Web Server, Database, and the Robotics

Operating System (ROS). Microcontrollers can be modularly added to the system via USB and automatically configured as ROS nodes through negotiations.

- **Mobile Application**

D: The mobile application is designed such that the user can receive push notifications and see various sensor data during the brew process.

2 Project Specifications

This section outlines the functional and non-functional requirements of the project design.

2.1 Functional Specifications

Table 2.1 describes each functional requirement and highlights whether it is essential or not to the completion of the project.

Table 1: An overview of each functional specification of the project

Specification	Classification	Description
Completion of the Brewing Process	Essential	The device automatically completes the brewing process, consisting of these steps: • Mash • Sparge • Boil Wort • Dispense Hops • Aerate the Wort • Pitch the Yeast • Kegging and Dispensing in its entirety and in the correct order. For more information on the brewing process and the terms used, please see the Glossary.
Heating Unit	Essential	Mashing the grains, sparging the grist, and boiling the wort all require high water temperatures. The system is able to accurately heat the contents to a minimum of 110°C within 1°C of error.

Cooling Unit	Essential	Yeast pitching and fermentation happen immediately after boiling the wort, but require temperatures a specific temperature (typically 20°C). The system is able to rapidly cool the wort to a temperature within 1°C of the target temperature so that the yeast may be pitched safely.
Temperature Regulation	Essential	The system is able to control temperature for each step in the brewing process. An error of 1°C is the target specification.
Trub Removal	Non-Essential	Remove the undesired sediment and other byproducts at the bottom of the fermenter so that ageing may take place within the vessel. The user does not have to maintain the trub accumulated by the brewing process.
Application Notifications	Non-Essential	The system provides notifications to the user updating them on the current state or action being taken. Additionally errors or warnings can be sent, in cases of emergency such as clogs, low oxygen or carbon dioxide, or an improper environment.
Database	Essential	The system is able query the specific steps it needs to automate for a specific brew. Additionally, it can store various data involved in brewing including temperature, pH levels and density measurements to generate logs and reports.
Reproducibility	Non-Essential	The system is able to record recipes, ingredients and steps for various brews. The user can recreate the same type of brew if they select one of the recipes.

2.2 Non-Functional Specifications

Table 2 describes each non-functional requirement and highlights whether it is essential or not to the completion of the project.

Table 2: An overview of each non-functional specification of the project

Specification	Classification	Description
Temperature Accuracy	Essential	The system accurately regulates the temperature required for the brewing process within 1°C of the targeted temperature.
Volume Control	Essential	The end result must be greater than or equal to the target yield volume. Target yields typically consist of 15.5 gallons, 7.75 gallons, and 5.16 gallons. These are the standard sizes of a half barrel keg, quarter barrel keg, and a sixth barrel keg respectively.
Size	Essential	The dimensions are limited such that the vessel can fit within a standard 36 inch residential door frame.
Mobility	Essential	The system is able to be relocated by a single person using the aid of caster wheels and or a standard 18 inch wide utility dolly.
Sanitation	Essential	The system employs SUS 304 stainless steel to maintain food-grade sanitation conditions. The vessel is self cleaning since a sanitary environment is crucial for the brewing process.

3 Detailed Design

3.1 Mechanical Design

The mechanical design component of this project is comprised of two aspects: the structural design of the fermenter and the thermodynamics of the heating and cooling systems. There are many sources for pre-fabricated fermenters for home brewing, but the custom nature of this device required the construction of a purpose-built tank, shown in Figure 2. The tank features a double walled construction to enclose the heating and cooling systems of the inner tank as well as provide thermal insulation.



Figure 2: Illustration of the double wall design for the fermenter

3.1.1 Material

To meet the requirements of food grade equipment, the components of the fermenter that will come into contact with the brewing ingredients are to be constructed from stainless steel. While the Canadian Food Inspection Agency (CFIA) (CFIA) does not outline specific details for the materials to be used in food processing, it states that, “Food contact surfaces of equipment and utensils are smooth, non-corrosive, non-absorbent, non-toxic, free from pitting, cracks or crevices, and able to withstand repeated cleaning and sanitation” [2]. To meet the non-corrosive guideline set by the CFIA, American Iron and Steel Institute (AISI) 304 stainless steel was the specific alloy of stainless steel chosen for the construction of the food-contacting surfaces of the fermenter. 304 is a member of the Austenitic Stainless Steel family (see Austenite), which have the best general corrosion resistance of any family of stainless steel. They also contain low levels of carbon, aiding in weld-ability. The low levels of carbon also act to reduce the generation of Inter-granular Corrosion caused by welding, over the life of the component [3]. Tables of the composition of 304 and pertinent mechanical properties are included in A. The outer tank of the fermenter will not come into contact with the brewing ingredients, and as such, other materials separate from stainless steel may be used. Aluminium was chosen to be the material for the outer tank due to its low mass and financial cost.

3.1.2 Tank Structure

When contemplating the mechanical design for a double-walled system such as this fermenter, the wall thickness for the inner tank should be determined early as this will dictate the structural support required between the inner and outer tanks, as well as the frame to support the entire vessel. Forced carbonation of

different brews can reach up to 30psi. The American Society of Mechanical Engineers (ASME) outlines a function for determining the wall thickness of a pressure vessel as

$$t = \frac{P_{work} \times r \times FS}{\sigma_{uts} \times E} \quad (1)$$

Where:

P is the working pressure of the vessel

r is the radius of the vessel

FS is the factor of safety

σ_{uts} is the Ultimate Tensile Strength (UTS) of the material

E is the non-dimensional efficiency of a welded seam

Therefore:

$$t = \frac{206.84kpa \times 254mm \times 9}{505MPa \times 0.6} \quad (2)$$

$$t = 1.560mm \quad (3)$$

The recommended thickness derived from this equation is approximately 1/16 of an inch, or 16 gauge when used to describe sheets of stainless steel. The outer tank thickness can now be specified using the mass of the inner tank and the brewing ingredients. The design of the system features collars at the top and bottom of both tanks to distribute the load evenly around the perimeter of the outer tank. The deflection and shear formulas used to determine the forces acting on the tank collars were first introduced in ME 220, Mechanics of Deformable Solids, and ME 322 Machine Design.



Figure 3: Illustration of the double wall design for the fermenter

Formula 4 was found using the Mechanical Engineer's Handbook [4]. The maximum stress experienced by the collar of the outer tank occurs at the inner edge of the collar. The UTS of aluminium can be used as this stress to determine the minimum sheet thickness required to support the fermenter.

$$\sigma = \frac{3w}{mt^2(a^2 - b^2)} \left(a^4(3m + 1) + b^4(m - 1) - 4ma^2b^2 - 4(m + 1)a^2b^2 \ln\left(\frac{a}{b}\right) \right) \quad (4)$$

Where:

t is the sheet thickness
m is the reciprocal of Poisson's Ratio
a is the outer diameter of the collar
b is the inner diameter of the collar
w is the applied force on the collar

Therefore:

$$t^2 = \frac{3w}{m\sigma(a^2 - b^2)} \left(a^4(3m + 1) + b^4(m - 1) - 4ma^2b^2 - 4(m + 1)a^2b^2 \ln\left(\frac{a}{b}\right) \right) \quad (5)$$

The resultant thickness is 2.05mm, or 0.08in. Accounting for a safety factor, a sheet thickness of 0.125 in. was selected to meet the requirements for the outer tank.

3.2 Electrical System

The electrical system consists of two main parts, the controller and the power electronic circuitry. These two subsystems are coupled closely together in that the control signals produced by the controller are amplified by the power electronics and sent to each plant. The main plants that are controlled and powered by the electrical systems are the heating element, cooling system, mixing motor, and various small similar actuators.

3.2.1 Controller Designs

Of the four controllable subsystems, the heating and cooling for fine temperature regulation is the highest priority for robust design. The mixing and actuators are necessary for the final realized system, but can easily be constructed with off-the-shelf components and minimal design effort. The actuators used in the system to open/close various valves and chutes are implemented using servomotors as the electrical component. The controllers used to control these actuations outputs a simple on-off signal and does not require feedback in the form of sensors. The continuous-time transfer function model of a DC motor with motor voltage as input and gear angle as output is shown Equation 6

$$P(s) = \frac{K}{s(\tau s + 1)} \quad (6)$$

which is open-loop unstable when tracking a step input. However, when tracking a ramp (i.e. a constant velocity) the system is stable, though with large steady-state error. This makes intuitive sense because when a constant voltage is applied to a DC motor, the speed of the motor stabilises to a constant value but the position of the motor goes unstable and increases indefinitely. Because of the fact that we don't require any sort of motor position tracking, nor do we require precise speed control, the control system for the mixing DC motor is an open-loop system without any feedback. Instead, the gear ratio of the gearbox that is powered by this motor is constructed to allow us crude speed control by simply changing the voltage output to the motor. Since the temperature model of the main volume of water is the same regardless of increasing or decreasing temperature, with only a sign-change to account for, the cooling system is assumed to function identically to the heating system in terms of power delivered to or taken from the water. Figure 4 shows a sketch of the physical system, and the equation that relates heat input to heat storage and heat loss is

described in 7.

$$Qh = Qs + Ql \quad (7)$$

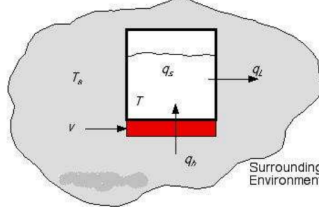


Figure 4: A heat system diagram for a closed volume of liquid

If we differentiate with respect to time, we get Equation 8 [5].

$$P = C \times \frac{dT}{dt} + k(T - T_a) \quad (8)$$

where P is the power input into the system, C is the thermal capacity of the water, k is the thermal conductance of the tank, T_a is the ambient temperature of the room, T is the temperature of the water, and t is time. Taking the Laplace transform of both side, we get Equation 9.

$$P(s) = C \times s \times T(s) + kT(s) \quad (9)$$

resulting in Equation 10

$$\frac{T(s)}{P(s)} = \frac{1}{Cs + k} \quad (10)$$

our transfer function model of the heating of a volume of liquid. Finding the value of the parameter C is relatively straightforward, it is equal to the mass of water in the system multiplied by the specific heat capacity of water. Since our system is designed to hold 50L of water, we get the result shown in 11

$$C = 50L \times 1 \frac{kg}{L} \times 4200 \frac{J}{kgK} = 210,000 \frac{J}{K} \quad (11)$$

Since the inner tank of our conical vessel is well insulated, we know that the value of k (in units of Watts per Kelvin) is small. However, calculating k mathematically is difficult and it is easier to instead perform some system identification on the completed system. The controller used to control the temperature of the volume of water can be implemented in many different ways, however we are deciding to look at 2 main options: a Proportional-Integral-Derivative (PID) controller, and a hysteresis-thermostat controller. Pictured below in Figure 5 and Figure 6 are Simulink models created in MATLAB to simulate these two possible controller schemes. Additionally, the tuned PID parameters used in the latter diagram, as calculated using the built-in Simulink PID Tuner tool-kit, are listed in Figure 7.

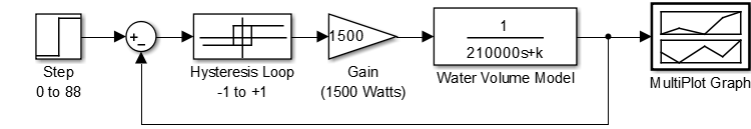


Figure 5: Hysteresis loop temperature controller and plant model

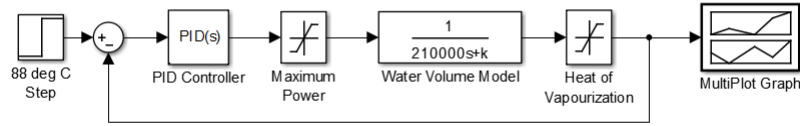


Figure 6: PID temperature controller and plant model

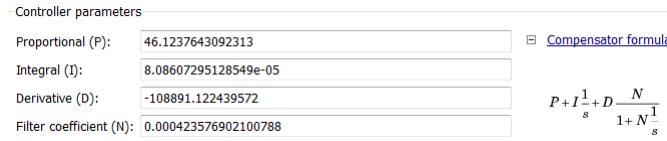


Figure 7: PID controller parameters

In these models, the step input has an amplitude of 88 degrees Celsius, simulating the heating of water from 10°C to 98°C; and the maximum allowable control signal is 1500W, which equates to 12.5A on standard 120V household power. We are looking to optimise a number of parameters with the selection of one of these control systems. Specifically, we want the best combination of settling time and steady-state error versus cost and complexity of implementation. Pictured below in Figure 8 and Figure 9 are the step responses of each of these systems, simulated assuming a k-value of zero (well insulated, lossless vessel).

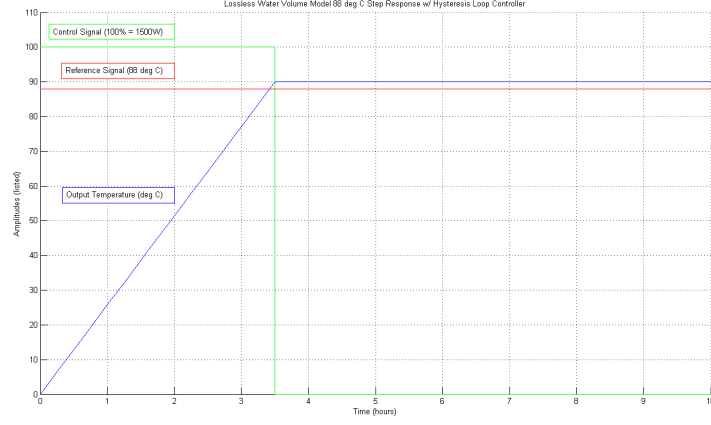


Figure 8: Lossless hysteresis loop control step response with control signal

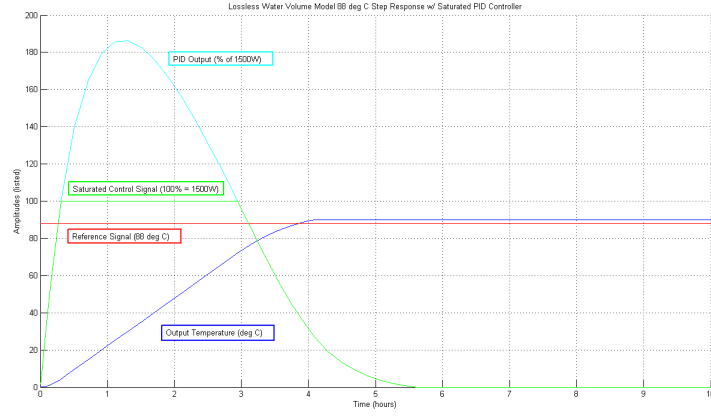


Figure 9: Lossless PID control step response with control signal

It can be seen from these plots that the hysteresis controller reaches the full-value thirty minutes earlier than the saturated PID controller. Since this model is perfect and lossless, the operation of the hysteresis loop isn't very apparent. If we increase the thermal conduction value to a modest level ($k = 10$), we get the response plotted in Figure 10. In this figure, we can clearly see that the switching period is around 20-minutes, which is long enough to ensure that we aren't over-switching our relays to run this controller. The new settling time is close to five hours in this plot.

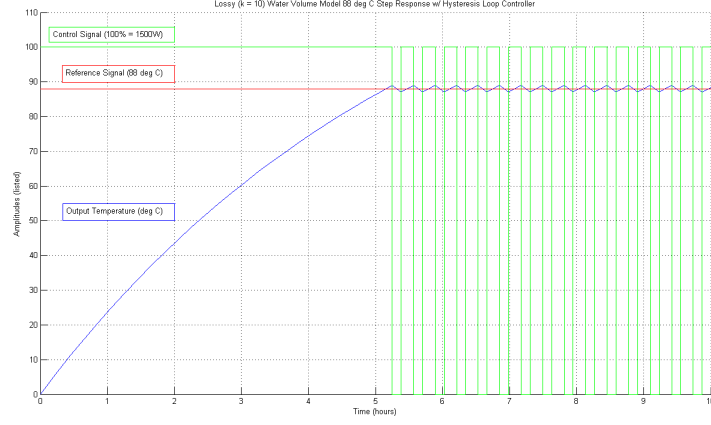


Figure 10: Lossy hysteresis loop step response with control signal

We can compare this to Figure 11 which plots the response of the PID system with varying k -values. In this plot, we see that for many values of k , the response time is also close to 5 hours, but as k increases the steady-state error quickly worsens to an unacceptable level.

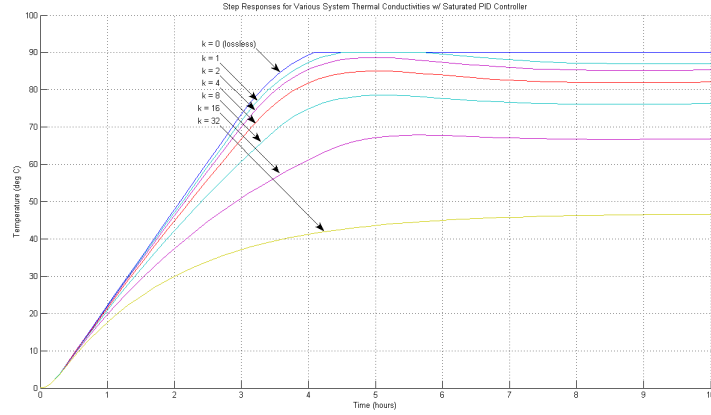


Figure 11: PID controller step response for various thermal conductivities

For the PID system, the “critical k -value”, defined as the k -value for which the steady-state error is beyond an acceptable level, is below $k = 4$. If we compare this to Figure 12 where the critical k -value of 16.67 for the hysteresis controller is plotted, we can see that the hysteresis controller is much more robust when faced with uncertain values of k since the critical value is over four times larger.

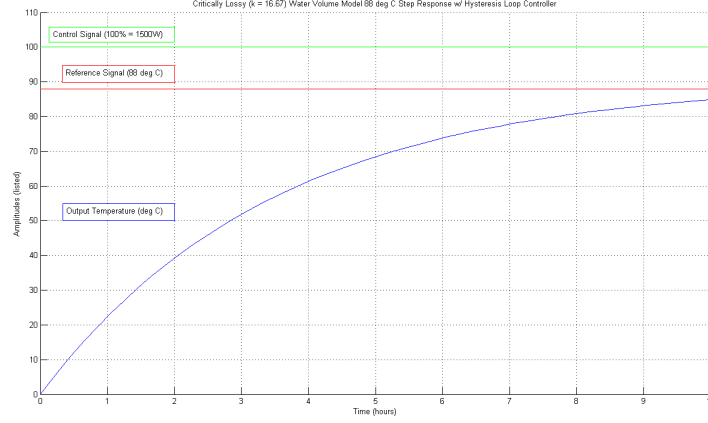


Figure 12: Critically lossy hysteresis loop step response with control signal

As such, the hysteresis-loop thermostat controller is being chosen to implement the heating and cooling of the vessel. Additional to the superior performance, the hysteresis controller has the added benefit of reduced implementation complexity in terms of the required power electronic circuitry, as discussed in the next section.

3.2.2 Power Electronics Design

The purpose behind the design of the power electronic circuitry is to be able to effectively implement the control strategies required to meet our design criteria. With the chosen control systems discussed in the previous section, the basic design of the power electronic circuits is straightforward. Depending on the size and power requirements of the servomotors needed to actuate the mechanical chutes and valves of the system, one of two circuits can be implemented. For large servos, the circuit pictured in Figure 13 can be implemented to power the motor.

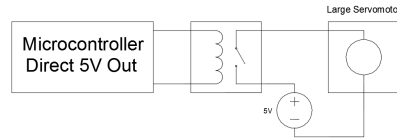


Figure 13: Large Servo Motor Power Circuit

However, for smaller servos that require less current to function, the relay and external DC source can be removed from the circuit completely, and the motor can be powered directly from the microcontroller's 5V output pins, as depicted in Figure 14.

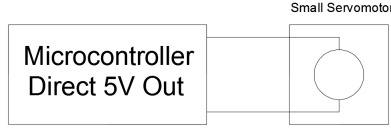


Figure 14: Small Servo Motor Power Circuit

The heating element alluded to in the previous section can easily be realised by a length of electrically insulated resistive wire, which can be coiled and wrapped around the base of the conical vessel. The specifics of wire itself (resistivity, thickness, length) are arbitrary by themselves, but they must be selected to yield a total resistance of 9.6σ . That value corresponds to a current of $12.5A$ being forced by $120V$, which in turn equates to $1500W$. The $1500W$ value is chosen so that the entire system can be powered by a standard $15A$ household circuit without tripping the breaker, plus a safety margin so that other small components can be powered while the main heaters are engaged. Figure 15 and Figure 16 illustrate the circuits for the hysteresis control and the unselected PID control schemes respectively. The Hysteresis control is realised with a simple circuit consisting of a relay between the $120V$ power supplied by the house and the 9.6 Ohm resistive heater load. The PID controller could have been realised with a triac whose firing angle is varied to modulate the power output to the heating coil.

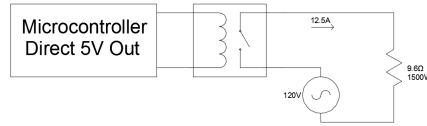


Figure 15: Hysteresis Loop Controller Power Circuit

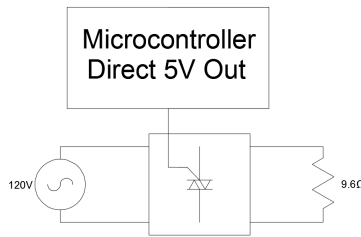


Figure 16: PID Controller Power Circuit

Both the mechanical mixer and the cooling system are driven by the same component; a DC motor. The circuit used to power the DC motor is pictured in Figure 17. It consists of a step-down transformer to reduce the mains voltage down to $12V$ to power the motors, then a full-bridge thyristor rectifier is used to simultaneously rectify the AC input voltage to a DC voltage and control the magnitude of the output voltage by varying the thyristor firing angle. The DC motor can be modelled as an inductance in series with a resistance, followed by a reverse-emf voltage.

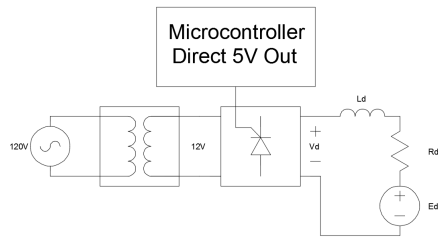


Figure 17: Thyristor Bridge Motor Power Circuit

Only the mechanical mixing motor requires voltage control to control the speed of the mixing. Since the cooling system operates on a hysteresis control scheme, the motor used to pump refrigerant only needs to be turned on or off. As such, the power circuitry for this system could be modified to use a simpler diode bridge rectifier circuit and a relay instead of the thyristor bridge, however for the purpose of bulk component shipment it is easier to build 2 identical thyristor bridge rectifier circuits to power both large DC motors.

3.3 Computer System

The computer systems consist of three main components, the embedded central computer, the microcontroller and sensor pairs, as well as a Web Server and Database pair to allow for storage and communications. The detailed description of each subcomponent and the methods of interaction between them is discussed in this section.

3.3.1 Embedded Computer

The primary function of the embedded computer system is to aggregate data in close to real-time from the multiple sensors used in the design. Analysis of this data is crucial for determining the current progress in the mash, sparge, and boiling of the wort. For specific analysis of sensor data please see subsection 3.3.2.

For ease of repeatability, availability, and cost the Raspberry Pi was selected. At the time of design and creation of this report, this is the most recent hardware revision of the Raspberry Pi. The relevant hardware specifications for the Raspberry Pi is a quad-core Acorn Reduced Instruction Set Computing (RISC) Machine (ARM) processor, 1GB of Random Access Memory (RAM), and a Bluetooth dongle for local wireless communications (rated to 50m). Since the outer shell of the fermenter is aluminum, the casing for the Raspberry Pi can act as a heat sink, allowing it to be safely overclocked to 1GHz, 500MHz, and 500MHz for the ARM processor frequency, sdram frequency, and L2 cache (core) frequency respectively. The configuration file below allows for a substantial performance boost with negligible thermal impact.

```
arm_freq=1000
sdram_freq=500
core_freq=500
over_voltage=2
arm_freq_min=400
sdram_freq_min=250
```

The operating system chosen was the ARM build of Ubuntu 14.04 Long Term Support (LTS), codename Trusty Tahr, as well as Robot Operating System (ROS) Jade Turtle for sensor coordination and communication. Ubuntu 14.04 was chosen over the default Raspbian operating system for increased security due to the use of SELinux policies. Since the Raspberry Pi will be acting as a LAMP server as well for mobile interaction and remote notifications, enforcing read only access over the configured port of choice is a necessity. For an in depth overview of the mobile application please see subsection 3.4. Additionally, the LTS version of Ubuntu was chosen for a maximum of five years support, as well as increased compatibility and reliability for repository packages specific to the trusty distribution.

3.3.2 Sensors and Microcontrollers

Every sensor in the system is paired with a microcontroller to enable a modular design and a plug and play like support. By adding more sensors to the system, more information can be provided as feedback to enable better logging and regulation of the brewing environment. The only mandatory sensors for the system are a main temperature sensor for the wort and flow meters on the liquid inputs and outputs of the system. By introducing additional sensors, such as pH and density sensors, certain yield thresholds of interest in the sparge and fermentation procedure can be met rather than approximated. Additionally, this will allow for a tier based configuration for the automated vessel to reduce cost and complexity if desired. As the end goal for this project is to have a community influenced input, having a customizable sensor configuration with open hardware and software pairs is encouraged. For maximum affordability the Arduino compatible Teensy-LC microcontroller is used to poll the sensors and report data, however any Arduino compatible microcontroller may be used. The bridge between the ROS instance on the Raspberry Pi and the Arduino compatible microcontroller is achieved through the use of the ROSSerial package. Example code for obtaining data from a TMP102 temperature sensor over I2C at address 0x91 [6] can be seen below [7].

```
#include <Wire.h>
#include <ros.h>
#include <std_msgs/Float32.h>

//Set up the ros node and publisher
std_msgs::Float32 temp_msg;
ros::Publisher pub_temp("temperature", &temp_msg);
ros::NodeHandle nh;

int sensorAddress = 0x91 >> 1; // From datasheet
long publisher_timer;

void setup()
{
    Wire.begin();          // join i2c bus (address optional for master)
    nh.initNode();
    nh.advertise(pub_temp);
}
```

```

void loop()
{
  if (millis() > publisher_timer) {
    // step 1: request reading from sensor
    Wire.requestFrom(sensorAddress,2);
    delay(10);
    if (2 <= Wire.available()) // if two bytes were received
    {
      byte msb;
      byte lsb;
      int temperature;

      msb = Wire.read(); // receive high byte (full degrees)
      lsb = Wire.read(); // receive low byte (fraction degrees)
      temperature = ((msb << 4); // MSB
      temperature |= (lsb >> 4); // LSB

      temp_msg.data = temperature*0.0625;
      pub_temp.publish(&temp_msg);
    }
    publisher_timer = millis() + 1000; //publish once a second
  }
  nh.spinOnce();
}

```

Additionally, controllers can be linked to the system via Arduino compatible microcontrollers and receive inputs based on sensor data gathered via ROS publications. To improve the resolution or accuracy of data and controls, the polling interval of each of the microcontrollers can be calibrated appropriately. Since most sensors will remain idle for a majority of the brewing process, their sampling rates can be scaled using a simple algorithm as suggested in the code sample below.

```

#include <std_msgs/Float32.h>

int sample_rate = MIN_SAMPLE_RATE;
int sample_count = 0;

void scale_rate(std_msgs::Float32 prev, std_msgs::Float32 curr)
{
  if(prev == curr)
  {
    sample_count++;
  }
  else
  {
    sample_count--;
  }
}

```

```

if(sample_count == SCALE_THRESHOLD)
{
    sample_rate = increase_rate();
    sample_count = SCALE_THRESHOLD >> 1;
}
else if(sample_count == 0)
{
    sample_rate = decrease_rate();
    sample_count = SCALE_THRESHOLD >> 1;
}
}

```

The two fundamental controllers that require communications for the operation of the system are the water controller and the heating and cooling controllers. Figure 18 demonstrates a state diagram for the water controller and Figure 19 demonstrate the state diagrams for toggling the heating and cooling states of the temperature controller. The variables volume, temp, target and offset represent the current volume, current temperature, desired resultant, and amount of allowed deviation from target respectively.

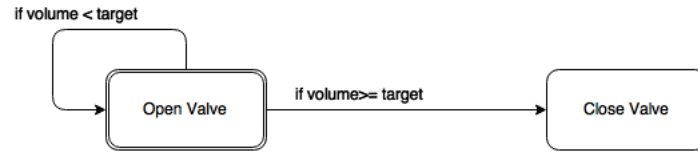


Figure 18: State diagram for the water controller component

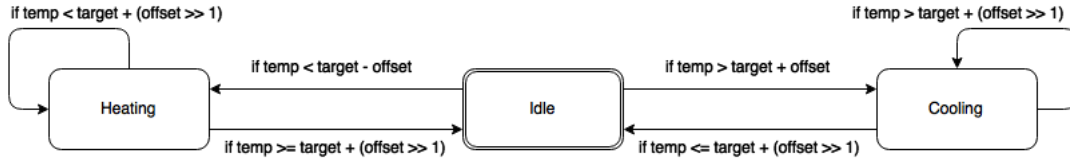


Figure 19: State diagram for the temperature controller component

3.3.3 Web Server and Database

A Lamp Apache MySQL PHP (LAMP) server is the current solution for delivering information from the embedded computer to the mobile application. All communications and parameters are to be communicated using a JavaScript Object Notation (JSON) format. The database will consist of relational connections between recipes, instructions, ingredients, sensors, and logged data as demonstrated in Figure 20.

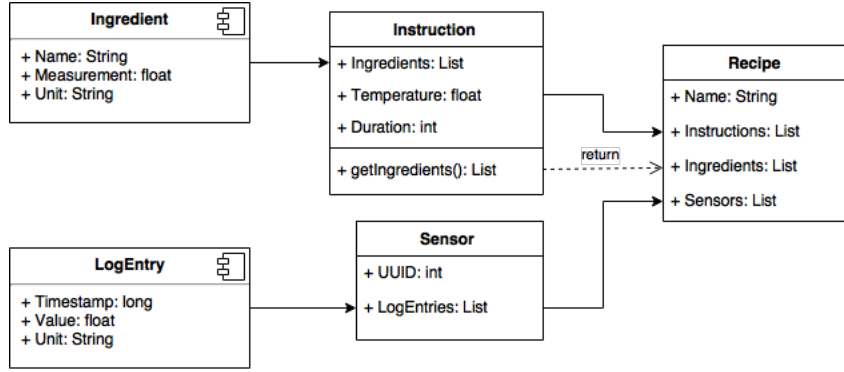


Figure 20: A UML diagram showing the relation between tables and their entries in the logging database

An important component of the web server is the server to client communication. The communication will be done through using the Google Cloud Messaging (GCM). This service allows a server to send messages to clients on various platforms [8]. There are two types of messages that can be sent using GCM, notification and data messages. The differences between the notification and data messages is that GCM automatically displays the notification messages on the client's behalf. Where as the data messages has to be parsed by the client in order to be used. Another difference is while the notification type messages has fixed key value pairs, data type messages can have custom ones. Both message types are in JSON format, which is shown below.

```

{
  "to" : "XYZ",
  "data" : {
    "degrees" : "85",
    "component" : "Water"
  }
}
  
```

The “to” value holds the unique application id and the data block holds the key value pairs that are encapsulated in the message. For message passing, this application uses the data message type. This is because notification messages does not offer custom key-value pairs that is required for transferring information from server to client. Although additional processing is done in the client side, data messages can be flexible to handle the various state changes that the system may undergo. In conclusion, the GCM component of the web server uses the Extensible Messaging and Presence Protocol (XMPP) protocol that sends data type messages to the client.

3.4 Mobile Application

The mobile application subsystem provides an interface to the user and consists of three main components. The first is a service that provides real time data of the current brew to the user, the second component is an interface that provides data analytics recorded by the sensor, as well as providing an interface that allows the user to input controls.

3.4.1 Real Time Data

This component of the application provides data from the brew system to the user's device. There are main two methods that the application provides data to the user. The first method is to display the current state of the brew in system through push notifications. Whenever a change is detected in the vessel such as temperature change, process change or any other state changes there might be, there would be a notification on the client's device to let the user know. The second method of the component is to provide interface that provides data statistics based on sensor data recorded by previous brews. This section will focus on the design for both methods. Figure 21 demonstrates the design flow for the push notification service.

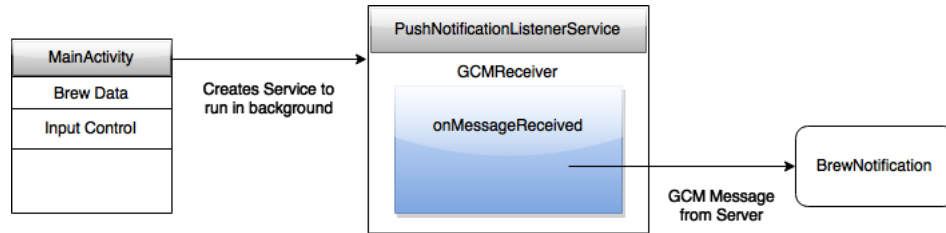


Figure 21: Flow Design for Notification Service

MainActivity: This activity starts on application launch (when user taps on the app icon). The activity will start the **PushNotificationListenerService** to run in the background. **PushNotificationListenerService:** The Listener uses the **GCMReceiver** class to listen for and to receive the data messages from the server. Whenever a JSON message arrives to the client side, the **onMessageReceived** function is called to parse the JSON message. **onMessageReceived:** This function parses the JSON message received by the Listener and creates a type of **BrewNotification** to display to the user. **BrewNotification:** This object will encapsulate notification created for the end user. Types of **BrewNotification** are based on priority: max, medium and low. Max priority notification is given to messages that relay critical information about the brew system (overheat, pressure build up, etc). Medium priority notifications are created for important messages (brew changing state, brew completed). Low priority notifications are given to messages that contain messages which the user does not necessarily have to know (boil temperature reached). To summarize, once the server sends a message to the client application, the Listener service will process the message through the **onMessageReceived** function. Once the function parses which type of message it is, it displays the appropriate **BrewNotification** to the user. The next part of the component displays previous data statistics recorded by the sensors on previous brews. The data displayed includes temperature readings, volume, sugar concentration and other sensor measurement. This gives the user the ability to look at past brews and possibly make adjustments. Figure 22 illustrates the application flow for the data statistics.

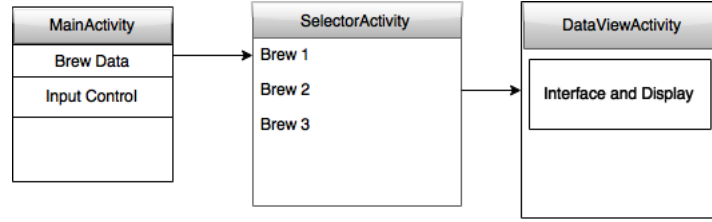


Figure 22: Flow Design for Notification Service

MainActivity: The main activity (same from the previous component) allows the user to launch the activity to select previous brews. **SelectorActivity:** This activity contains a selectable list of all previously done brews. By selecting one item launches the **DataViewActivity** based on the brew. **DataViewActivity:** Displays various data (temperature, pressure, yield percentage) of the brew selected by the user. Providing this information allows the user to view and experiment with previous brews to enhance the quality of future brews. In conclusion, the real time data component of the application is designed to keep the user notified about the brew system, whether that would be about the current state of the brew process or about previous brew data. This enhances user experience such that the user does not have to constantly check the brew process to receive information since the system informs the user.

3.4.2 Input Controls and Interaction

The second component of the application is to allow the user to control the brew system with the mobile application. The goals of this component is to allow the user to start the brew process through the application and to provide control over various conditions within the brew process such as the maximum temperature, the target sugar concentration and others that would affect the brew's quality. Figure 23 below shows the design flow for this component.

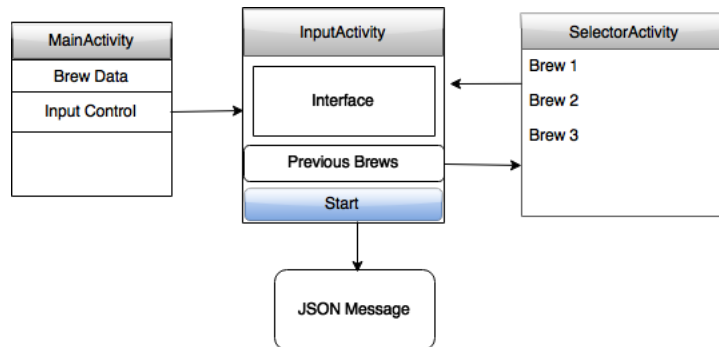


Figure 23: Flow Design for Notification Service

MainActivity: Option to select the input control activity, the main activity first checks if the mobile device is on the same local network as the vessel. If the device is not on the same network, the MainActivity will not allow the user to select the “Input Controls” option. This is because the mobile application is designed to allow users to perform write applications only when connected on the same network as the web server, otherwise the application allows read-only operations. Although this will limit the application's

usability, it ensures a high level of security on the brew system, where it does not allow others to exploit the application over the internet to cause the main vessel to malfunction. **InputActivity:** The InputActivity allows the user to select a previous brew process to modify or create a brand new brew process. The user can change various states such as max temperature for boils, number of hours and other conditions that might affect the brew. The user is also able to start the brew from the interface, by tapping the START button snapped to the bottom of the interface. **SelectorActivity:** If the user selects a previous brew, the activity retrieves the relevant data from the server, and populate the interface of the InputActivity. If the user taps the START button in the InputActivity interface, the application creates a JSON file with all the data mapped to a key value. The client will send the JSON file to the server and wait for a response. An error response from the server will have another key explaining the details of the error. The errors are based on lack of ingredients in the vessel or a malfunction detected by the system. A success response from the server means that the system started the brew.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

The final design meets the objective and specifications listed in the project specification document. The combination of the subsystems creates an automated brew system that can control the climate within the vessel. These controls are brought about by the inclusion of a temperature regulation which requires real-time temperature feedback to maintain precision. There is also an additional specification to create a user interface so that the brewing device may be operated via a mobile device.

4.2 Use of Advanced Knowledge

The subsystem designs all require upper year knowledge to complete. For developing the communications portion of the embedded computer system SELinux policies enforcing read only access is a crucial security precaution. Proper security enforcement can minimize the potential for malicious attacks on the system as outlined in ECE 458, Computer Security. Developing the Web Server and the Mobile Application includes knowledge from such courses as ECE356 (Database Systems), ECE454 (Distributed Systems) and ECE452 (Software Design and Architectures). Designing databases requires knowledge of various schemas, relations, normalization and indexing that are involved in the ECE356 contents. Designing a mobile application involves the knowledge of software design processes, methods and implementing those designs that is taught in ECE452. Finally, the brew system requires various sensor networks and communication between a centralized server to multiple clients which involves knowledge from ECE454. The design and analysis of the control systems used in the electrical subsystems require knowledge of control systems that are taught in both ECE380 and ECE 481, and the design and analysis of the power electronic components uses concepts take directly from ECE463.

4.3 Creativity, Novelty, Elegance

The novelty of our design is that it involves so many components from various Engineering backgrounds to complete. The system allows the user to have precise controls of the brew but at the same time offer flexibility in various environmental conditions during the process. One of the most elegant part of the project

is that it takes the entire brew process and compacts it into one single vessel. The use of a double walled tank enables the heating and cooling apparatus to be mounted inside the device itself, adding to the singular design aspect of the fermenter.

4.4 Student Hours

The group focus for the past 2 months was on the initial phase of the project. This involves designing components, ordering parts and determining an appropriate timeline. All of the group members met once a week to discuss progress, plans and future deliverables. Table 3 indicates the total amount of hours each group member has worked on the project.

Table 3: Total Hours

Group Member	ID	Hours
██████████	██████████	69
██████████████████	██████████	57
██████████	██████████	63
██████████	██████████	53

4.5 Potential Safety Hazards

The power electronic systems pose many hazards to the building of this system. The custom 120V circuitry is inherently dangerous to test, and proper precautions must be taken to avoid electrocution. The heating components of the system are capable of getting very hot, so proper care must be taken to avoid burns and potentially starting a fire. The motors can be very dangerous if anyone puts their hand near the spinning parts while the system is live, so it is important to make sure that everything is powered down and disconnected before attempting to adjust any of the motor actuated assemblies.

4.6 Project Time-Line

Due to amount of design required for each of the subcomponents, the time-line has been split into individual Gantt charts for Mechanical Design, Electrical Systems, Computer Systems, and Mobile Development shown in Figures 24, 25, 26, and 27 respectively. Please note that each of these Gantt charts have overlapping dates.

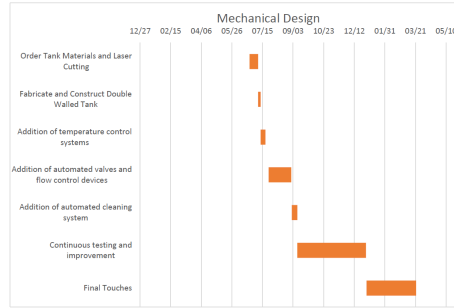


Figure 24: Gantt Chart for Mechanical Subcomponent Time-Line

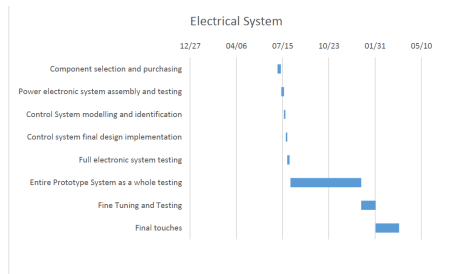


Figure 25: Gantt Chart for Electrical Subcomponent Time-Line

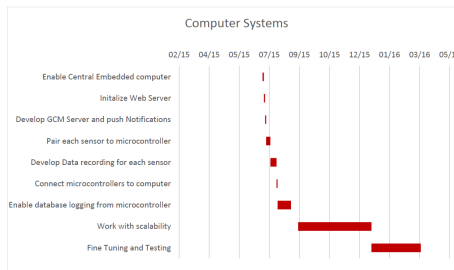


Figure 26: Gantt Chart for Computer Subcomponent Time-Line

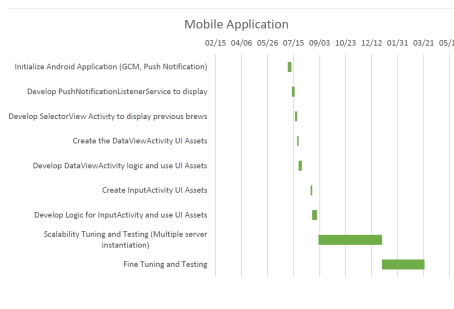


Figure 27: Gantt Chart for Mobile Development Subcomponent Time-Line

Acronyms

AISI American Iron and Steel Institute.

ARM Acorn RISC Machine.

CFIA Canadian Food Inspection Agency (CFIA).

GCM Google Cloud Messaging.

I2C Inter-Integrated Circuit.

JSON JavaScript Object Notation.

LAMP Lamp Apache MySQL PHP.

LTS Long Term Support.

PID Proportional-Integral-Derivative.

RAM Random Access Memory.

RISC Reduced Instruction Set Computing.

ROS Robot Operating System.

USB Universal Serial Bus.

UTS Ultimate Tensile Strength.

WLAN Wireless Local Area Network.

XMPP Extensible Messaging and Presence Protocol.

Beer Terminology

Grist The combination of milled grains to be used in a particular brew. Also sometimes applied to hops [9]..

Mash (Verb) To release malt sugars by soaking the grains in water. (Noun) The resultant mixture [9]..

Pitch To add yeast..

Sparge To spray grist with hot water in order to remove soluble sugars (maltose). This takes place at the end of the mash [9]..

Trub The layer of sediment that appears at the bottom of the fermentation vessel upon the completion of fermentation..

Wort The solution of grain sugars strained from the mash tun [9]..

Technical Terminology

Austenite A non magnetic crystal structure of iron. May exist in solid solution with the aid of alloying elements..

Austenitic Stainless Steel An alloy family of stainless steels high in Austenite phase grain structure..

Inter-granular Corrosion Corrosion caused by varying composition between the grains and grain boundaries within a metal. Welding 304 stainless steel may lead to intergranular corrosion if not cooled properly..

Raspberry Pi 2 Model B The Raspberry Pi 2 Model B is the second generation Raspberry Pi. It replaced the original Raspberry Pi 1 Model B+ in February 2015..

Ultimate Tensile Strength The maximum tensile stress a material can experience before breaking or fracturing.

Yield Stress The stress at which a material begins to plastically deform..

References

- [1] A. Tepedelen, "History of homebrewing," 2013. <http://allaboutbeer.com/article/power-to-the-people/>. [Accessed 2015-05-19].
- [2] C. F. I. Agency, "General principles of food hygiene, composition and labelling," 2014. <http://www.inspection.gc.ca/food/non-federally-registered/safe-food-production/general-principles/eng/1352919343654/1352920880237?chap=3>. [Accessed 2015-07-01].
- [3] C. Matthews, *ASME Engineer's Data Book, 2nd Ed.* East Lansing, MI: ASME Press, 2001.
- [4] M. Kutz, *Mechanical Engineer's Handbook, 3rd Ed. Book 1.* Mississauga, ON: Wiley, 2005.
- [5] D. J. J. G. Haresh A. Suthar, *Modelling and Analysis of the Simple Water Heater System.* Vadodara, India: International Journal of Electrical and Computer Engineering (IJECE), 2011.
- [6] T. Instruments, "Tmp102," 2015. <http://www.ti.com/product/tmp102>. [Accessed 2015-07-01].
- [7] ROS, "Measuring temperature," 2014. http://wiki.ros.org/roserial_arduino/Tutorials/Measuring%20Temperature. [Accessed 2015-07-01].
- [8] Google, "Use google cloud messaging (gcm)," 2015. <https://support.google.com/googleplay/android-developer/answer/2663268?hl=en>. [Accessed 2015-07-01].
- [9] BeerAdvocate, "Beer & brewing terminology," 2015. <http://www.beeradvocate.com/beer/101/terms/>. [Accessed 2015-05-30].
- [10] E. Oberg, *Machinery's Handbook 28th Edition.* South Norwalk, CT: Industrial Press Inc, 2008.
- [11] M. LLC, "Aisi type 304 stainless steel," 2000. <http://asm.matweb.com/search/SpecificMaterial.asp?bassnum=MQ304A>. [Accessed 2015-07-01].

A Appendix

This section contains important lookup values for mechanical and materials calculations.

Table 4: Alloying elements of AISI 304 Stainless Steel [10]

Alloying Element	Cr	Ni	C	Mn	Si	P	S	N
Wt. %	18-20	8-10.5	0.08*	2.0*	0.75*	0.045*	0.030*	0.10*

* Indicates maximum

Table 5: AISI Type 304 Stainless Steel Mechanical Properties [11]

Yield Stress	UTS	Elasticity	Poisson Ratio	Hardness	Conductivity	Heat Capacity
215 MPa	505 MPa	193 MPa	0.29	123	16.2 W/m-K	0.5 J/g°C

Table 6: Alloying elements of 5052 Aluminium [10]

Alloying Element	Cr	Cu	Fe	Mn	Mg	Zn	Si
Wt. %	0.15-0.35	0.1*	0.04*	0.1*	2.2 2.8	0.1*	0.25*

* Indicates maximum

Table 7: Type 5052 Aluminium Mechanical Properties [11]

Yield Stress	UTS	Elasticity	Poisson Ratio	Hardness	Conductivity	Heat Capacity
193 MPa	228 MPa	70.3 MPa	0.33	60	138 W/m-K	0.88 J/g°C